InsightAI - 多模态多智能体拟真面试考评系统方案详细设计

一、 技术栈选型

1. 后端底层架构 (Backend)

- **核心框架**: Java 17 + Spring Boot 3.2
- **AI 编排引擎**: **LangChain4j**（业内最主流的 Java 大模型开发框架，实现对大模型的标准接入、ChatMemory 上下文记忆管理和 RAG 流程抽象）
- **流式通信协议**: **SSE (Server-Sent Events)**（基于 HTTP 长连接的单向服务端推送机制，比 WebSocket 更轻量，完美适配大模型的“打字机”流式输出）
- **持久化层**: MySQL 8.0 + MyBatis-Plus（存储无模式的 JSON 考评数据与基础用户信息）
- **安全认证**: JWT (JSON Web Token) 无状态鉴权

2. 前端架构与端侧智能 (Frontend & Edge AI)

- **核心框架**: Vue 3 (Composition API) + Vite + Vue Router
- **UI 组件库**: Element Plus
- **端侧 AI 引擎 (多模态感知)**: `face-api.js`（基于 TensorFlow.js，纯浏览器端运行的人脸检测与表情识别模型，实现“零延迟、无侵犯”的情感采集）
- **原生多媒体通信**:
  - **Web Speech API** (实现 STT 实时语音转文本、TTS 面试官语音合成变声)
  - **WebRTC API** (`getUserMedia` 调取前置摄像头媒体流)
- **底层图形渲染**: 原生 **HTML5 Canvas 2D API**（用于开发高保真的 六维雷达图、数据热力图、性能极高的主页粒子动画）

二、 核心业务模块剖析

模块 A：全双工多模态面试终端 (Video Interview Mode)

- **业务逻辑**：候选人开启摄像头和麦克风，与 AI 面试官进行“面对面”交流。
- **技术实现**：
  - **VAD (Voice Activity Detection) 模拟算法**：前端通过定时器（`setTimeout`）监听语音输入流的静音片段。一旦超过 2.5 秒未发声，自动切断录音并向后端发送消息，实现无缝的“对话交锋”，省去手动点击发送的繁琐。
  - **角色化 TTS 变声**：拦截 AI 流式返回的文本，提取角色信息（技术官用男声、HR用女声），调用系统内置语音引擎进行音色匹配与动态朗读。

模块 B：Agent 集群与自适应考评引擎 (Multi-Agent & Adaptive Engine)

- **业务逻辑**：告别单一的“一问一答”机器人，模拟真实大厂的交叉面试场景。
- **技术实现**：
  - **轮转状态机**：通过记录对话轮次（Round），动态注入不同的 System Prompt。前段由“面试组长”引导，中段交由“技术面试官”硬核施压，尾段转给“HR”考察软素质。

- **自适应难度控制**：在技术官的 Prompt 中注入“降维打击与底层深挖”规则。AI 需要在上下文中判断候选人的准确率，动态收缩或拔高下一道题的难度。

模块 C：隐式情感监控系统 (Implicit Emotion Analytics)

- **业务逻辑**：捕捉求职者在回答压力测试题时的微表情变化，反映真实的心理门槛。
- **技术实现**：
  - 面试全程画布隐藏采样：每隔 3 秒调用 `requestAnimationFrame` 从 `video` 标签中截取帧，输入给端侧微调的神经网络（face-api.js），输出实时情绪（Happy, Sad, Fearful 等）。
  - 算法聚合汇总：在面试结束时，将成百上千次的情感切片聚类洗散，加权计算出全局的**“自信指数 (Confidence Score)”**。**全程数据不上传服务器以保护隐私**，符合 GDPR 规范。

模块 D：多维可视化诊断报告 (Visual Diagnostic Report)

- **业务逻辑**：面试结束不仅给出文字，更要生成结构化的评级数据。
- **技术实现**：
  - 大模型 JSON 强制约束：在 `InterviewServiceImpl` 结尾的 Prompt 中，通过 Few-shot 强制 AI 输出严格匹配前端解析库的 `JSON Schema`（包含 6A 维度的 A-E 评级和 3 条提升建议）。
  - Canvas 热力图溯源：在面试历史界面取出历史 JSON 数据，通过原生 Canvas 绘制能力演进的高密热力图（Heatmap），颜色深浅（蓝色透明度）直观反映候选人的成长轨迹。

三、 难点突破

难点 1：流式传输 (SSE) 在前后端的拼接与乱码问题

- **痛点**：大语言模型吐字是一个 Token 一个 Token 进行的，Spring Boot 原生的 HTTP 不支持这种长时间单挂协议，且前端如果每次来一个字就全量渲染 Markdown，会导致页面 CPU 满载（卡顿）。
- **解决方案**：
  1. 使用了 Spring Boot 的 `SseEmitter` 维护长连接，并自定义了 UTF-8 的 MediaType，解决了 Spring 拦截器将半截中文字符强制截断导致的乱码问题。
  2. 前端维护了一个局部 `currentAiText` 的响应式 Proxy 代理池，只有遇到标点符号或特定的延时周期，才调用 `marked.js` 将 Markdown 转化为 HTML，极大地**降低了 DOM 频繁重绘 (Repaint/Reflow) 带来的性能损耗。**

难点 2：AI 判断面试终止导致的“流中断”异常

- **痛点**：由于是多轮连续对话，我们需要一个条件来结束面试生成报告。如果靠用户点击“结束”太生硬，如果靠 AI 自然结束，前端如何精准捕捉大模型的意思？
- **解决方案**： 创造了**“特殊隐藏标记法”。**在后端 Prompt 中严令要求，如果 AI 判定面试该结束了，必须在最后输出隐式标记符 `[TERMINATE]`。 前端在处理 SSE 数据流时，采用正则表达式动态侦测，一旦捕获到该标记，立刻中断长连接，截断该非法字符不展示给用户，并唤醒前端报告生成逻辑，实现了前后端**无侵入的协议握手**。

难点 3：浏览器端视频 AI 带来的内存泄露

- **痛点**：一边要处理 WebSocket/SSE 通信，一边要在纯前端跑 `face-api.js` 人脸识别，时间久了浏览器内存暴增（OOM 崩溃）。
- **解决方案**：

1. **时间维度的降采样 (Time-domain Downsampling)**：从原本每秒 60 帧的实时检测，降采样为通过定时器每 3000ms 截取一帧进行运算。这使得情感数据的置信度保持在 90% 以上，但 CPU 消耗骤降 95%。
2. **模型 CDN 动态分发**：避免将庞大的 `.weights` 权重文件打包到自己服务器中，利用 jsDelivr 的边缘节点动态拉取张量模型，极大压减了项目的首屏白屏时间。